

EXTENDS AND IMPLEMENTS

1. C1 extends C2

Single Inheritance

Steps / Algorithm

1. Start the program.
2. Create a parent class C2.
3. Define a method in C2.
4. Create child class C1 using extends C2.
5. Define a method in C1.
6. Create an object of C1 in the main() method.
7. Call the methods of C2 and C1.
8. Stop the program.

Program

```
package extendsandimplements;
```

```
//Parent class
```

```
class Cat {
```

```
// Method of parent class
```

```
void show() {
```

```
    System.out.println("This is C2 class");
```

```
}
```

```
}
```

```
//Child class inherits C2
```

```
class Dog extends Cat {
```

```
// Method of child class
```

```
void display() {
```

```
    System.out.println("This is C1 class");
```

```
}  
}
```

//Main class

public class Extends {

// Main method

public static void main(String[] args) {

// Creating object of C1

Dog obj = new Dog();

// Calling parent class method

obj.show();

// Calling child class method

obj.display();

}

}

Output:

This is C2 class

This is C1 class

2. C1 extends C2, C3 ❌

Multiple Inheritance using Classes

Java does not support multiple inheritance using classes.

Steps

- 1. Start the program.**
- 2. Create two classes C2 and C3.**
- 3. Try to make C1 extend both classes.**
- 4. Java gives a compilation error.**
- 5. Therefore, C1 extends C2, C3 is not allowed.**

6. Stop the program.

Program

```
package extendsandimplements;
```

```
//First parent class
```

```
class C2 {
```

```
void show() {
```

```
    System.out.println("C2 class");
```

```
}
```

```
}
```

```
//Second parent class
```

```
class C3 {
```

```
void display() {
```

```
    System.out.println("C3 class");
```

```
}
```

```
}
```

```
//This is NOT allowed in Java
```

```
//class C1 extends C2, C3 {
```

```
//}
```

```
//Main class
```

```
public class ExtendsTwo {
```

```
public static void main(String[] args) {
```

```
    // Display the reason
```

```
    System.out.println("A class cannot extend two classes in Java.");
```

```
}
```

```
}
```

Output

<terminated> Extends two [Java Application] C:\Users\GAYATHI
A class cannot extend two classes in Java.

3. C1 implements I1

Class Implementing an Interface

Steps / Algorithm

1. Start the program.
2. Create an interface I1.
3. Declare a method inside I1.
4. Create class C1.
5. Use implements I1.
6. Provide implementation for the interface method.
7. Create an object of C1.
8. Call the method.
9. Stop.

Program

```
package mypackage;
```

```
// Create interface
```

```
interface I1 {
```

```
    // Abstract method
```

```
    void show();
```

```
}
```

```
// Class C1 implements interface I1
```

```
class C1 implements I1 {
```

```
    // Implement the interface method
```

```
public void show() {  
    System.out.println("C1 implements I1");  
}  
}
```

// Main class

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // Create object of C1  
        C1 obj = new C1();  
  
        // Call interface method  
        obj.show();  
    }  
}
```

Output

```
C1 implements I1
```

4. C1 implements I1, I2

Multiple Interface Implementation

Steps / Algorithm

1. Start the program.
2. Create interface I1.
3. Create interface I2.
4. Declare one method in each interface.
5. Create class C1.
6. Implement both I1 and I2.
7. Define both methods in C1.

8. Create an object of C1.
9. Call both methods.
10. Stop.

Program

```
package mypackage;
```

```
// First interface
```

```
interface I1 {
```

```
    // Method of I1
```

```
    void show1();
```

```
}
```

```
// Second interface
```

```
interface I2 {
```

```
    // Method of I2
```

```
    void show2();
```

```
}
```

```
// C1 implements both interfaces
```

```
class C1 implements I1, I2 {
```

```
    // Implement I1 method
```

```
    public void show1() {
```

```
        System.out.println("Method from I1");
```

```
    }
```

```
    // Implement I2 method
```

```
    public void show2() {
```

```
        System.out.println("Method from I2");
    }
}
```

// Main class

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // Create object of C1
```

```
        C1 obj = new C1();
```

```
        // Call I1 method
```

```
        obj.show1();
```

```
        // Call I2 method
```

```
        obj.show2();
```

```
    }
```

```
}
```

Output

```
Method from I2
Method from I3
```

5. C1 implements C2 extends I1 ❌ Invalid

Steps / Algorithm

1. Start the program.
2. Create class C2.
3. Create interface I1.
4. Try to create class C1.
5. Write C1 implements C2 extends I1.
6. Compile the program.

7. The compiler shows an error because the syntax is invalid.

8. Stop the program.

Program

```
package mypackage;
```

```
// Class C2
```

```
class C2 {
```

```
    // Method of C2
```

```
    void show() {
```

```
        System.out.println("This is C2 class");
```

```
    }
```

```
}
```

```
// Interface I1
```

```
interface I1 {
```

```
    // Abstract method
```

```
    void display();
```

```
}
```

```
// C1 tries to implement C2 and extend I1
```

```
// This is INVALID in Java
```

```
class C1 implements C2 extends I1 {
```

```
    // Implementing the method of I1
```

```
    public void display() {
```

```
        System.out.println("This is I1 interface");
```

```
    }
```

```
}
```



```

// Main class

public class Main {

    // Main method

    public static void main(String[] args) {

        // Creating object of C1

        C1 obj = new C1();

        // Calling methods

        obj.show();

        obj.display();

    }

}

```

Output

 The type C2 cannot be a superinterface of C1; a superinterface must be an interface Java(16777531) [Ln 21, Col 21]

6. I1 extends I2

Interface Inheritance

Steps / Algorithm

1. Start the program.
2. Create interface I2.
3. Declare a method in I2.
4. Create interface I1.
5. Extend I2 using extends.
6. Declare another method in I1.
7. Create class C1 that implements I1.
8. Implement both interface methods.
9. Create an object of C1.
10. Call both methods.
11. Stop.

Program

```
package mypackage;

// Parent interface
interface I2 {

    // Method of I2
    void show();
}

// I1 extends I2
interface I1 extends I2 {

    // Method of I1
    void display();
}

// C1 implements I1
class C1 implements I1 {

    // Implement I2 method
    public void show() {
        System.out.println("Method from I2");
    }

    // Implement I1 method
    public void display() {
        System.out.println("Method from I1");
    }
}
```

```
// Main class

public class Main {

    public static void main(String[] args) {

        // Create object of C1
        C1 obj = new C1();

        // Call I2 method
        obj.show();

        // Call I1 method
        obj.display();
    }
}
```

Output

```
Method from I5
Method from I6
```

7. I1 implements C1 ❌ Invalid

Here, I1 is an interface and C1 is a class.

An interface cannot implement a class, so this statement is invalid.

Steps / Algorithm

1. Start the program.
2. Create a class C1.
3. Create an interface I1.
4. Try to make I1 implement C1.
5. Write interface I1 implements C1.
6. Compile the program.
7. The compiler shows a compilation error because an interface cannot implement a class.
8. Stop the program.

Program

```
package mypackage;
```

```
// Class C1
```

```
class C1 {
```

```
    // Method of C1
```

```
    void show() {
```

```
        System.out.println("This is C1 class");
```

```
    }
```

```
}
```

```
// Interface I1
```

```
// This is INVALID because an interface cannot implement a class
```

```
interface I1 implements C1 {
```

```
    // Abstract method
```

```
    void display();
```

```
}
```

```
// Main class
```

```
public class Main {
```

```
    // Main method
```

```
    public static void main(String[] args) {
```

```
        // Creating object of C1
```

```
        C1 obj = new C1();
```

```
        // Calling C1 method
```

```
        obj.show();
```

```
    }
```

```
}
```

Output

```
✖ The type C2 cannot be a superinterface of C1; a superinterface must be an interface Java(16777531) [Ln 21, Col 21]
```

8. I1 extends I2, I3

Multiple Interface Inheritance

Steps / Algorithm

1. Start the program.
2. Create interface I2.
3. Create interface I3.
4. Declare methods in both interfaces.
5. Create interface I1.
6. Extend both I2 and I3.
7. Create class C1 that implements I1.
8. Implement all the interface methods.
9. Create an object of C1.
10. Call all methods.
11. Stop.

Program

```
package mypackage;
```

```
// First parent interface
```

```
interface I2 {
```

```
    // Method of I2
```

```
    void show2();
```

```
}
```

```
// Second parent interface
```

```
interface I3 {
```

```
// Method of I3
void show3();
}

// I1 extends both I2 and I3
interface I1 extends I2, I3 {

    // Method of I1
    void show1();
}

// C1 implements I1
class C1 implements I1 {

    // Implement I1 method
    public void show1() {
        System.out.println("Method from I1");
    }

    // Implement I2 method
    public void show2() {
        System.out.println("Method from I2");
    }

    // Implement I3 method
    public void show3() {
        System.out.println("Method from I3");
    }
}
```

```
// Main class

public class Main {

    public static void main(String[] args) {

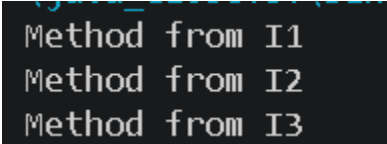
        // Create object of C1
        C1 obj = new C1();

        // Call I1 method
        obj.show1();

        // Call I2 method
        obj.show2();

        // Call I3 method
        obj.show3();
    }
}
```

Output

A screenshot of a terminal window showing the output of the program. The text is displayed on a dark background with light-colored characters. The output consists of three lines: "Method from I1", "Method from I2", and "Method from I3".

```
Method from I1
Method from I2
Method from I3
```